CS 621 — Quiz 2 Study Notes

Spark Structured Streaming

Complete Exam Preparation Guide

- Module 6.1 — Streaming Data Concepts & Structured Streaming
- Module 6.2 — Aggregations, Time Windows, Watermarks & Joins

Scalable AI Services | Spark Streaming | Databricks

■ Table of Contents

■ MODULE 6.1 — Introduction to Spark Structured Streaming

◆ 1. Why is Stream Processing Getting Popular?

The vast majority of data in the world is **streaming data** — it is generated continuously as events happen over time. Three major reasons drive stream processing adoption:

Driver	Explanation
Data Velocity & Volumes	Rising data volumes require continuous, incremental processing — you cannot batch-process everything on a schedule.
Real-time Analytics	Businesses need fresh data for actionable insights and faster, better decisions.
Operational Applications	Critical apps (fraud detection, IoT, monitoring) need real-time data for instantaneous response.

◆ 2. Stream Processing Use Cases

Stream processing is a **key component** of big data applications across all industries:

Use Case	Description
Notifications	Alert users or systems when specific events occur in real-time.
Real-time Reporting	Live dashboards with up-to-the-second metrics.
Incremental ETL	Continuously load and transform new data as it arrives.
Update Data to Serve in Real-time	Keep serving databases fresh for low-latency lookups.
Real-time Decision Making	Fraud detection, recommendation systems, risk scoring.
Online ML	Apply machine learning models to streaming events as they arrive.

◆ 3. Advantages of Stream Processing (vs. Batch)

Advantage	Explanation
Incremental Processing	Natural and intuitive — process only new data, not entire datasets.
Low Latency SLAs	Unlocks millisecond-to-second response times for time-sensitive workloads.
Better Fault-Tolerance	Checkpointing means the system can resume from where it left off after restarts.
Higher Compute Utilization	Continuous, incremental processing is more efficient than large batch jobs.

◆ 4. Modern Stream Processing Architecture

A modern streaming pipeline has 5 layers that data flows through:

Layer	Description
1. Stream Data Sources	DBMS/CDC, Clickstreams, App Events/Logs, IoT devices — data originates here.
2. Stream Data Ingestion	Streaming data lands in a message bus, e.g. **Apache Kafka**.
3. Stream Processing Engine	**Spark Structured Streaming** — applies transformations: window aggregation, pattern detection, enrichment, routing.
4. Data Storage Layer	**Delta Lake** — transactional, versioned storage for processed results.
5. Query Engine	Serves real-time analytics, triggers and alerts to end consumers.

Key insight: Data is *continuously, incrementally processed* as it appears — not in scheduled batches.

◆ 5. Challenges of Stream Processing

Stream processing is NOT easy. Common challenges include:

- Processing each event **exactly once** despite machine failures
- Processing **out-of-order data** based on application timestamps (event time)
- Maintaining **large amounts of state** in memory efficiently
- Handling **load imbalance and stragglers** (slow tasks that delay window computations)
- Determining how to **update output sinks** as new events arrive
- Writing data **transactionally** to output systems

A straggler is an event/task in a distributed pipeline that falls behind expected processing time. Even if most tasks finish quickly, a few slow ones delay the entire system because Spark must wait for all partitions before triggering watermarks, window computations, joins, or aggregations. This causes increased latency, out-of-order results, or backpressure. Solution → Spark Structured Streaming handles all of these challenges automatically.

◆ 6. What is Spark Structured Streaming?

Property	Detail
Definition	A scalable, fault-tolerant stream processing framework built on Spark SQL engine.
API	Uses existing structured APIs (DataFrames, SQL Engine) — same API as batch processing.
Stream Features	End-to-end exactly-once processing, fault-tolerance, checkpointing.
Unified API	Write nearly identical code for both batch and streaming — only read/write mode changes.

◆ 7. Streams as Unbounded Tables

Spark's mental model: treat a **data stream as an infinitely growing table**. New data arriving in the stream = new rows appended to the table.

Unbounded Table Property	Meaning
Never stops growing	New rows keep arriving — there is no predefined end.

Unbounded Table Property	Meaning
Append-only (conceptually)	New events are new rows; the table only grows forward.
Queries run incrementally	Results update as new rows appear, not by reprocessing all data.
Unlike a batch (static) table	Batch tables are finite; unbounded tables are theoretically infinite.

◆ 8. How Structured Streaming Works — The Trigger Loop

Behind the scenes Spark runs a **trigger loop** (micro-batch or continuous):

- **Step 1 — Check Source:** Look at Kafka, Delta, or files for new data since last trigger.
- **Step 2 — Fetch New Data:** Read the new rows and treat them as appended to the unbounded table.
- **Step 3 — Apply Query Logic:** Run your transformations, joins, and aggregations on the new micro-batch.
- **Step 4 — Write to Sink:** Persist results to Delta Lake, Kafka, files, etc.
- **Step 5 — Update Offsets / Checkpoint:** Save progress so the stream can resume exactly where it left off.
- **Step 6 — Sleep → Repeat:** Wait for the trigger interval, then start again.

> **File source behaviour:** When using files (JSON, CSV, Parquet) as input, Spark *watches a directory* and performs incremental file listing — it only reads NEW files since the last trigger, not the entire directory.

◆ 9. Anatomy of a Streaming Query — Core Concepts

Every streaming query has 4 main parts: **Source** → **Transformations** → **Sink** → **Options**

■ 9a. Source

Defines *where* to read data from. Returns a Spark DataFrame — common API for batch and streaming.

Source	How to Use	Notes
Kafka	spark.readStream.format("kafka").option("subscribe","topic").load()	Built into OS Spark
Files (JSON/CSV/Parquet)	spark.readStream.format("json").load("/path")	Watches directory incrementally
Delta Lake	spark.readStream.format("delta").load("/delta/path")	Databricks optimized
Event Hubs / Kinesis	Connector libraries	Databricks runtime only

■ 9b. Transformations

Any standard DataFrame/SQL operations applied to the streaming DataFrame — 100s of built-in SQL functions (e.g., **from_json**, **selectExpr**, **withColumn**).

- Example: Cast Kafka bytes → string → parse as JSON → generate nested columns

■ 9c. Sink

Defines *where* to write results. The sink receives incremental updates from the results table.

Sink	Use Case
Delta Lake (.format('delta'))	Production — persistent, versioned, queryable
Files (.format('parquet'/'csv'))	Production — file output

Sink	Use Case
Kafka (.format('kafka'))	Production — downstream streaming consumers
Console (.format('console'))	Development & debugging ONLY
Memory (.format('memory'))	Development & debugging ONLY

■ 9d. Checkpoint Location

A folder path where Spark saves the **progress of the streaming query** — which data has been processed and what offsets have been committed. **Required for fault-tolerance and exactly-once semantics.**

■ 9e. DataFrame — The Unifying API

Batch API	Streaming API
spark**.read**	spark**.readStream**
[.withWatermark(...)] — N/A	.withWatermark(...) — for late data handling
.table('source')	.table('source')
.withColumn() / .select()	.withColumn() / .select()
spark**.write**	spark**.writeStream**
[.outputMode(...)]	[.trigger(...)] [.queryName(...)]
.saveAsTable('sink_table')	.option('checkpointLocation',...).toTable('sink')

◆ 10. Trigger Types

The **trigger** defines how frequently Spark checks for new data and processes a micro-batch.

Trigger Type	Syntax	Behaviour
Fixed Interval Micro-batch	.trigger(processingTime="2 minutes")	Runs every 2 minutes. Most common for production workloads.
Triggered — One-time (single batch)	.trigger(once=True)	Process ALL available data as ONE micro-batch, then stop automatically.
Triggered — One-time (multi batch)	.trigger(availableNow=True)	Process ALL available data as MULTIPLE micro-batches, then stop. Better resource control than once=True.
Continuous Processing	.trigger(continuous="2 seconds")	EXPERIMENTAL — long-running continuous tasks with checkpoints at specified frequency. Very low latency.
Default	(no trigger specified)	Databricks: 500ms fixed interval. OS Spark: process each micro-batch immediately after previous finishes.

Exam tip — availableNow vs once: Both process all available data then stop. The difference: once=True uses a single micro-batch (can be huge); availableNow=True uses multiple smaller micro-batches for better memory control and parallelism.

◆ 11. Output Modes

Defines **how results are written to the sink** after each trigger. The supported modes depend on the query type.

Mode	What Gets Written	Typical Use Case
Append (most common)	Only NEW rows added since last trigger are written to the sink. Cannot be used with aggregations (unless watermarked).	Simple ingest / ETL with no aggregations
Complete	The ENTIRE updated result table is written to the sink on every trigger. Used for aggregations.	groupBy + count/sum across all time
Update	Only rows that were NEW or UPDATED since the last trigger are written. Most efficient for aggregations.	Aggregations where only changed results matter

Key rule: Append mode cannot be used with stateful aggregations without watermarks (results change over time so they can't be 'appended'). Complete mode rewrites everything — use carefully for large result tables.

◆ 12. Streaming from Delta Lake (Delta as Source)

Delta Lake has deep integration with Structured Streaming:

- **How it works:** Each *committed version* of the Delta table represents new data to stream. Delta transaction logs identify each version's new data files.
- **Append-only assumption:** Structured Streaming assumes append-only sources. Any non-append changes (UPDATE, DELETE, OVERWRITE) to a Delta table cause streaming queries to **throw exceptions**.

■ Enforcing Append-Only with Table Properties

```
ALTER TABLE my_table
SET TBLPROPERTIES ('delta.appendOnly' = 'true');
```

Operation	Allowed when appendOnly=true?
INSERT	■ Yes — allowed
UPDATE	■ No — throws error
DELETE	■ No — throws error
OVERWRITE	■ No — throws error

- For non-append changes, use **Delta Lake Change Data Feed (CDF)** to propagate arbitrary change events to downstream consumers.

■ Rate Limiting with DataStreamReader Options

Option	Default	Effect
maxFilesPerTrigger	1,000 files	Maximum files read per micro-batch — hard limit.
maxBytesPerTrigger	No default	Soft limit on data read per micro-batch.

◆ 13. Streaming to Delta Lake (Delta as Sink)

- Each micro-batch written to a Delta table is committed as a **new version**.
- Delta Lake as sink supports both **Append** and **Complete** output modes.

- **Append** (most common): new rows from each micro-batch are added to the table.
- **Complete**: replaces the entire table with each micro-batch — used for streaming aggregations.

> Append-only tables are ideal for: immutable log files, IoT data events, clickstream events, financial tick data, CDC raw bronze layer ingestion. These tables grow monotonically without mutation.

■ MODULE 6.2 — Aggregations, Time Windows, Watermarks & Joins

◆ 1. Types of Stream Processing — Stateless vs. Stateful

Type	Definition	Examples	Complexity
Stateless	Each record is processed independently. No dependency on previously seen records.	Data ingest (map-only), simple dimensional joins, filtering	Low
Stateful	Previously seen records CAN influence how new records are processed. State must be maintained across micro-batches.	Aggregations over time, fraud/anomaly detection, streaming joins, deduplication	High

Stateful processing is the key challenge of stream processing. Maintaining state requires memory and introduces complexity around late data, state expiry, and fault-tolerance.

◆ 2. How Stateful Streaming Works — The Intermediate State

Consider a simple **groupBy + count** query. On each trigger Spark:

- Step 1: Reads new incoming data (e.g., [cat, dog]).
- Step 2: Updates the **Intermediate State** in memory (running counts per key).
- Step 3: Writes the result rows to the **output sink** (in Complete mode: entire updated table).
- Step 4: Saves the **offset + intermediate state** to the **checkpoint** for fault-tolerance.

Trigger	Incoming Data	Intermediate State After	Output (Complete)
t=1s	cat, dog	cat:1, dog:1	cat:1, dog:1
t=2s	cat, dog, cat	cat:2, dog:1	cat:2, dog:1
t=3s	cat, dog, cat, owl	cat:3, dog:2, owl:1	cat:3, dog:2, owl:1

Code pattern: spark.readStream.<source>.groupBy('animal').count().writeStream.mode('complete').<sink>.trigger('1s').start()

◆ 3. Reasoning About Time — Event Time vs. Processing Time

Time Type	Definition	Why It Matters
Event Time	The time the event ACTUALLY OCCURRED — embedded in the data by the source device/application.	Gives correct business meaning. A transaction at 12:01 should count in the 12:00–12:10 window regardless of when Spark receives it.
Processing Time	The time Spark actually PROCESSES the record — when it arrives at the engine.	Can differ significantly from event time due to network delays, buffering, etc. Using processing time for aggregations gives incorrect results with delayed data.

Critical: Always use event time for business-meaningful aggregations. Processing time gives correct results only when data arrives in perfect order with no delay — which never happens in real systems.

◆ 4. Time-Based Windows — Tumbling vs. Sliding

Property	Tumbling Window	Sliding Window
Overlap	No overlap — discrete, non-overlapping intervals	Windows overlap — intervals share time
Event membership	Each event belongs to EXACTLY ONE window	Each event can belong to MULTIPLE windows
Parameters	windowDuration only	windowDuration + slideDuration
Visual pattern	[0–10], [10–20], [20–30] ...	[0–10], [5–15], [10–20] ...
Primary goal	Segmenting time into distinct blocks	Tracking trends over time (rolling KPIs)
When to use	Hourly/daily reporting, billing cycles	Moving averages, rolling counts

Sliding window rule: when **slideDuration < windowDuration**, windows overlap. If slideDuration == windowDuration, it behaves exactly like a tumbling window (just with optional offset).

◆ 5. The window() Function

Spark provides a built-in **window()** function that generates time window columns for groupBy aggregations.

Signature	Window Type	Example Output
window(timeCol, windowDuration)	Tumbling — single duration	[00:00–00:10], [00:10–00:20], ...
window(timeCol, windowDuration, slideDuration)	Sliding — overlapping windows	[00:00–00:10], [00:05–00:15], ...
window(timeCol, windowDuration, slideDuration, startOffset)	Offset tumbling/sliding — shift start time	[00:02–00:12], [00:12–00:22], ... (offset=2min)

■ Code Examples

Tumbling window (10 minutes, no overlap):

```
from pyspark.sql.functions import col, window

agg = (
    df
    .withWatermark('event_time', '15 minutes')  # recommended for state cleanup
    .groupBy(window(col('event_time'), '10 minutes'))  # tumbling: single duration
    .count()
)
```

Sliding window (10-minute window, slides every 5 minutes):

```
agg = (
    df
    .withWatermark('event_time', '15 minutes')
    .groupBy(window(col('event_time'), '10 minutes', '5 minutes'))  # sliding
```

```
    .count()
)
```

Offset tumbling window (aligned to 00:02 instead of 00:00):

```
agg = df.groupBy(
    window(col('event_time'), '10 minutes', '10 minutes', '2 minutes')
).count()
# windowDuration == slideDuration → still tumbling, just offset
```

◆ 6. Watermarking — Handling Late-Arriving Data

Watermarking solves two critical problems with stateful streaming:

- **Problem 1 — Memory pressure:** Without watermarks, Spark keeps ALL state indefinitely because any new event might match an old window. This eventually crashes the system.
- **Problem 2 — Late data:** Events can arrive long after they occurred. How long should Spark wait before finalising a window result?

■ How Watermarking Works

You define a **late threshold**. Spark tracks the **maximum event time seen so far** and computes:

Watermark = Max Event Time Seen – **Late Threshold**

Any event with *event time* < *watermark* is considered **too late** and MAY be dropped. Any event with event time ≥ watermark is **guaranteed** to be processed.

```
# Wait up to 10 minutes for late-arriving data
windowed_counts = (
    df
    .withWatermark('timestamp', '10 minutes')
    .groupBy(window('timestamp', '10 minutes'))
    .count()
)
```

Aspect	Guarantee
Within watermark threshold	Data WILL be processed — guaranteed.
Beyond watermark threshold	Data MAY be dropped — not guaranteed to be included.
State cleanup	Old state beyond watermark is cleared from memory — prevents OOM.
Output mode compatibility	Watermarking enables Append mode with aggregations (results only output when window is finalised).

Always use withWatermark() when doing windowed aggregations. Without it, state grows unboundedly and the query will eventually fail with OOM (out-of-memory) errors. The watermark tells Spark when it is safe to discard old state.

◆ 7. Memory Pressure — Solutions

Without bounded state, intermediate aggregation state grows indefinitely as the stream runs. GC pauses get longer and longer until failure is inevitable.

Solution	Approach	How to Enable
Watermarking	Define a late threshold so old state can be purged from memory.	.withWatermark('event_time', '10 minutes')
RocksDB State Store	Save intermediate state OFF-HEAP using RocksDB instead of executor memory. Reduces GC pressure significantly.	spark.conf.set("spark.sql.streaming.stateStore.providerClass", "com.databricks.sql.streaming.state.RocksDBStateStoreProvider")

RocksDB is an embedded key-value store. When used as the state store, it keeps state on local disk (off-heap), drastically reducing JVM memory usage and GC pauses for large stateful streaming queries.

◆ 8. Streaming Joins — Overview

Structured Streaming supports two types of join scenarios:

Join Type	Left Side	Right Side	State Required?
Stream-Static	Streaming DataFrame	Static DataFrame	No (Stateless)
Static-Stream	Static DataFrame	Streaming DataFrame	No (Stateless)
Stream-Stream	Streaming DataFrame	Streaming DataFrame	Yes (Stateful)

- **Key property:** The result of a streaming join is generated *incrementally* — exactly as if the stream data were a static table with the same rows.
- **Stream-stream joins** buffer past input from BOTH streams in state, because any new event from stream A might match a past event from stream B.

◆ 9. Streaming Joins — Full Support Matrix

Left Side	Right Side	Join Type	State?	Watermark Required?
Static	Static	All join types	No	No
Static	Stream	Inner Join	No (Stateless)	No
Static	Stream	Right Outer	No (Stateless)	No
Static	Stream	All other joins	■ Not supported	—
Stream	Static	Inner Join	No (Stateless)	No
Stream	Static	Left Outer	No (Stateless)	No
Stream	Static	Left Semi	No (Stateless)	No
Stream	Static	All other joins	■ Not supported	—
Stream	Stream	Inner Join	Yes (Stateful)	Optional
Stream	Stream	Left Outer	Yes (Stateful)	Mandatory on RIGHT
Stream	Stream	Left Semi	Yes (Stateful)	Mandatory on RIGHT
Stream	Stream	Right Outer	Yes (Stateful)	Mandatory on LEFT

Left Side	Right Side	Join Type	State?	Watermark Required?
Stream	Stream	Full Outer	Yes (Stateful)	Mandatory on at least ONE side

■ Managing Memory in Stream-Stream Joins

In a stream-stream join, state grows *indefinitely* because all past input must be buffered. To bound this state:

- **1. Define watermark delays on BOTH input streams** — so Spark knows when events are too late.
- **2. Add event-time constraints on the join condition** — so Spark knows the maximum time gap between matching events.

```
# Watermarks on both sides
impressions = impressions.withWatermark('impressionTime', '2 hours')
clicks      = clicks.withWatermark('clickTime', '3 hours')

# Event-time constraint on join condition
joined = impressions.join(
    clicks,
    expr("""
        clickAdId = impressionAdId AND
        clickTime BETWEEN impressionTime AND impressionTime + INTERVAL 1 HOUR
    """)
)
```

For full state cleanup in stream-stream joins: specify watermark on BOTH sides AND add event-time constraints. Without constraints, Spark cannot determine the matching window and state will still grow.

■ Quick Reference, Key Terms & Exam Tips

◆ Key Terms Cheat Sheet

Term	Definition
Structured Streaming	Scalable, fault-tolerant stream processing framework built on Spark SQL engine.
Unbounded Table	Mental model: a stream = an infinitely growing table. New events = new rows appended.
Trigger	Defines how often Spark checks for new data. Types: fixed interval, once, availableNow, continuous.
Output Mode	How results are written to sink: Append (new rows only), Complete (entire table), Update (new + updated rows).
Checkpoint Location	Directory where Spark saves query progress (offsets + state) for fault-tolerance and exactly-once processing.
Sink	Output destination for streaming results: Delta, Kafka, files, console (debug), memory (debug).
Event Time	Timestamp embedded in the data — when the event ACTUALLY OCCURRED at the source.
Processing Time	Timestamp when Spark actually processes the event — can be much later than event time.
Tumbling Window	Non-overlapping fixed-size time intervals. Each event belongs to exactly ONE window.
Sliding Window	Overlapping fixed-size windows. Each event can belong to MULTIPLE windows.
window()	Spark SQL function to create time windows for groupBy aggregations.
Watermark	A lateness threshold. Events older than watermark may be dropped; state older than watermark is cleared.
withWatermark()	API call to define a watermark on a streaming DataFrame.
Stateless Processing	Each record handled independently — no history needed. Example: filter, map, simple join.
Stateful Processing	Records depend on previously seen data. Requires maintaining state. Example: aggregations, joins.
Intermediate State	Running partial aggregation results kept in memory/checkpoint between triggers.
RocksDB State Store	Off-heap state storage using RocksDB — reduces GC pressure for large stateful queries.
Straggler	A slow task/partition in a distributed pipeline that delays overall processing.
delta.appendOnly	Table property that prevents UPDATEs, DELETEs, OVERWRITEs on a Delta table.

Term	Definition
Change Data Feed (CDF)	Delta feature to propagate UPDATE/DELETE/INSERT events to downstream streaming consumers.
maxFilesPerTrigger	DataStreamReader option to limit files read per micro-batch (default 1000).
maxBytesPerTrigger	DataStreamReader option to set a soft byte limit per micro-batch (no default).
Stream-Static Join	Join between streaming and static DataFrame — stateless, no watermark needed.
Stream-Stream Join	Join between two streaming DataFrames — stateful, watermarks often required.

◆ Common Exam Questions & Answers

Q: What happens if you do not set a watermark in a long-running windowed aggregation?

A: The intermediate state will grow unboundedly over time. GC pauses will increase, eventually causing the query to fail with out-of-memory errors. Watermarks are essential to bound state size.

Q: What is the difference between once=True and availableNow=True triggers?

A: Both process all available data and stop. once=True processes everything in a SINGLE micro-batch (may be very large). availableNow=True uses MULTIPLE smaller micro-batches for better memory control and parallelism.

Q: Can you use Append output mode with a groupBy aggregation?

A: Not directly. With a simple aggregation, Append mode is not supported because aggregate results can change as new data arrives. However, with watermarking, Spark can use Append mode — it waits until a window is finalised (past the watermark) before appending the final result.

Q: Which streaming joins REQUIRE a watermark?

A: For stream-stream joins: Left Outer requires watermark on RIGHT, Right Outer requires watermark on LEFT, Full Outer requires watermark on at least one side, Left Semi requires watermark on RIGHT. Inner join watermarks are optional but recommended.

Q: What is a straggler and why does it matter?

A: A straggler is a slow task/partition that falls behind. It matters because Spark must wait for ALL partitions before triggering window computations, watermarks, joins, and aggregations. This causes increased latency and can lead to backpressure.

Q: What is the difference between event time and processing time?

A: Event time is when the event actually happened (embedded in data). Processing time is when Spark receives and processes it. They differ due to network delays, buffering, etc. Event time is used for business-correct aggregations.

Q: What does delta.appendOnly = true do?

A: It prevents non-append modifications (UPDATE, DELETE, OVERWRITE) on a Delta table. Only INSERTs are allowed. This is important for streaming sources since Structured Streaming assumes append-only inputs.

Q: What is the purpose of the checkpoint location?

A: It stores the streaming query progress — which offsets have been processed and the intermediate state. This enables fault-tolerance: if the query restarts, it resumes exactly from where it left off, guaranteeing exactly-once processing.

◆ Code Patterns to Remember

■ Complete Streaming Query Pattern

```
spark.readStream
    .format('kafka')                          # or 'delta', 'json', etc.
    .option('kafka.bootstrap.servers', '...')
    .option('subscribe', 'my-topic')
    .load()
    .selectExpr('cast(value as string) as json')
    .select(from_json('json', schema).alias('data'))
    .writeStream
    .format('delta')
    .outputMode('append')
    .option('path', '/delta/output/')
    .option('checkpointLocation', '/checkpoints/my-query/')
    .trigger(processingTime='1 minute')
    .start()
```

■ Windowed Aggregation with Watermark

```
from pyspark.sql.functions import window, col

result = (
    events_df
    .withWatermark('event_time', '10 minutes')   # tolerate 10min late data
    .groupBy(
        window(col('event_time'), '10 minutes')  # tumbling 10-min window
    )
    .count()
    .writeStream
    .outputMode('append')                         # ok with watermark
    .option('checkpointLocation', '/ckpts/')
    .start()
)
```

■ Stream-Stream Join with Watermarks

```
impressions = (
    spark.readStream.format('delta').load('/impressions')
    .withWatermark('impressionTime', '2 hours')
)
clicks = (
    spark.readStream.format('delta').load('/clicks')
    .withWatermark('clickTime', '3 hours')
)

joined = impressions.join(
    clicks,
    expr("""
        clickAdId = impressionAdId
        AND clickTime BETWEEN impressionTime
                          AND impressionTime + INTERVAL 1 HOUR
    """)
)
```

◆ Final Exam Strategy

Focus on the WHY behind every concept: • WHY use watermarks? → Bound state size + handle late data. • WHY event time over processing time? → Correctness with delayed/out-of-order data. • WHY append-only for Delta streaming source? → Structured Streaming cannot handle non-append changes without CDF. • WHY use Complete output mode? → Required for aggregations that update previously output rows. • WHY does stream-stream join need watermarks? → To bound the buffered state from both streams. • WHY use RocksDB state store? → Moves state off-heap to reduce GC pressure for large stateful queries. • WHEN to use availableNow vs once? → Both stop after processing all data; availableNow is more memory-efficient.